

Sistema de Control de Acceso de Estacionamiento SICAE

Plan operativo por integrante: fases, actividades, obligaciones, entregables y cadena de trabajo

Proyecto Final - Tecnologías para la Integración de Soluciones

Documento de coordinación para equipo de 4 integrantes

Elemento	Definición para el equipo
Objetivo del documento	Convertir la ruta de trabajo general en una guía accionable por integrante, evitando bloqueos, retrabajo y tareas que no suman a la evaluación.
Criterio rector	Cada fase debe terminar con una pieza funcional, probada, documentada y útil para el siguiente integrante.
Producto esperado	Microservicios funcionales, bases independientes, JWT, comunicación REST, contenedores, Postman y documentación técnica con evidencias.

Regla de oro del equipo

Nada está terminado si no tiene: código funcional, prueba en Postman, evidencia, explicación breve y entrega clara para el siguiente integrante.

1. Enfoque general de trabajo

La estrategia consiste en organizar el proyecto como una cadena de producción. Cada integrante trabaja una parte del sistema, pero su entrega debe habilitar el avance del siguiente. Esto evita que el proyecto se convierta en cuatro trabajos aislados y facilita demostrar una solución integrada.

Cadena de trabajo	Responsabilidad principal	Resultado esperado
Integrante 1	Arquitectura base, AuthService, JWT, bcrypt y configuración inicial.	Sistema con autenticación funcional y token disponible para proteger los demás servicios.
Integrante 2	UserService, usuarios, roles, estatus, clave única y validaciones.	Usuarios administrables y endpoint de validación para que otros servicios consulten usuarios activos.
Integrante 3	VehicleService, vehículos, placas, estatus y límite de vehículos activos.	Vehículos administrables y endpoint de validación para que ParkingService valide placas y asociación.
Integrante 4	ParkingService, integración, entrada, salida, espacios, costos, Postman y documento final.	Flujo completo de estacionamiento funcionando y listo para demostración.

2. Principios obligatorios para todos

- Trabajar con arquitectura Controller - Service - Repository en cada microservicio.
- Proteger los servicios con JWT, salvo el login.
- No eliminar físicamente entidades principales; cambiar estatus cuando aplique.
- Evitar duplicados: usuarios, correos, usernames y placas.
- Probar casos exitosos y casos de error.
- Documentar evidencias reales de Postman y contenedores.
- No agregar interfaces gráficas ni funciones extra que no afecten la calificación.
- Cada integrante debe poder explicar técnicamente su servicio y sus validaciones.

3. Fase 0 - Preparación inicial del equipo

Objetivo: congelar la estructura del proyecto antes de programar, para que todos trabajen con la misma organización, puertos, nombres, rutas y datos de prueba.

Integrante	Actividades y obligaciones	Entregable individual	Entregable para el siguiente
Integrante 1	Definir estructura general del repositorio, nombres de servicios, puertos, paquetes base y arquitectura interna. Establecer criterio común de organización.	Estructura base del proyecto y acuerdos técnicos iniciales.	Carpetas y convenciones listas para que los demás creen sus servicios sin inventar otra organización.
Integrante 2	Revisar tablas de usuarios, roles, tipos de usuario y programas educativos. Identificar campos	Diccionario simple de campos del dominio de usuarios.	Campos confirmados para AuthService y UserService.

Integrante	Actividades y obligaciones	Entregable individual	Entregable para el siguiente
	obligatorios para registrar, editar y validar usuarios.		
Integrante 3	Revisar tablas de marcas, modelos y vehículos. Identificar campos obligatorios y relaciones lógicas con usuarios.	Diccionario simple de campos del dominio de vehículos.	Campos confirmados para VehicleService.
Integrante 4	Revisar tablas de cajones y movimientos. Identificar campos para entrada, salida, ocupación, tarifa, costo y tiempos.	Diccionario simple del dominio de estacionamiento.	Campos confirmados para ParkingService.

Decisión del equipo	Valor recomendado
AuthService	Puerto 8081
UserService	Puerto 8082
VehicleService	Puerto 8083
ParkingService	Puerto 8084
PostgreSQL usuarios	Puerto 5432
MySQL vehículos	Puerto 3307
MySQL estacionamiento	Puerto 3308

4. Fase 1 - Bases de datos y datos de prueba

Objetivo: importar los scripts entregados, confirmar tablas y dejar datos listos para probar casos exitosos y casos de error. Esta fase desbloquea a todos los integrantes.

Integrante	Actividades y obligaciones	Entregable individual	Entregable para el siguiente
Integrante 1	Importar la base de usuarios en PostgreSQL. Confirmar conexión, tablas principales y credenciales de prueba.	Base de usuarios funcionando y captura de conexión.	Usuario administrador activo para probar login y permisos.
Integrante 2	Crear o confirmar usuarios de prueba: administrador, usuario normal activo y usuario inactivo. Verificar roles y tipos de usuario.	Lista de usuarios de prueba y estatus de cada uno.	Credenciales válidas para AuthService y UserService.
Integrante 3	Importar la base de vehículos en MySQL. Confirmar marcas, modelos, vehículos y placas de prueba.	Base de vehículos funcionando y captura de conexión.	Vehículos asociados a usuarios existentes.
Integrante 4	Importar la base de estacionamiento en MySQL. Confirmar cajones libres, cajones ocupados y	Base de estacionamiento funcionando y captura de conexión.	Cajones y movimientos listos para probar entrada y salida.

Integrante	Actividades y obligaciones	Entregable individual	Entregable para el siguiente
	movimientos de prueba.		

Dato mínimo	Uso dentro del proyecto
Administrador activo	Login, registro de usuarios y cambio de estatus.
Usuario normal activo	Registro de vehículos y entrada al estacionamiento.
Usuario inactivo	Pruebas de rechazo.
Vehículo activo	Entrada válida.
Vehículo inactivo	Prueba de rechazo.
Placa repetida	Prueba de duplicidad.
Usuario con cuatro vehículos activos	Prueba del límite de vehículos.
Usuario con dos vehículos dentro	Prueba del límite de acceso al estacionamiento.
Cajón libre y cajón ocupado	Prueba de entrada, ocupación y errores.

5. Fase 2 - AuthService y seguridad base

Responsable principal: Integrante 1. Esta fase va primero porque todos los demás servicios dependen del token generado en el login.

Integrante	Actividades y obligaciones	Entregable individual	Entregable para el siguiente
Integrante 1	Crear AuthService con login, JWT, bcrypt, validación de usuario activo y conexión a PostgreSQL. Definir respuesta estándar de autenticación.	Login funcional, token generado, validación de credenciales y Dockerfile inicial.	Token válido, header de autorización y credenciales de prueba para los demás integrantes.
Integrante 2	Probar login con administrador, usuario normal e inactivo. Confirmar que el token funciona para UserService.	Capturas de login exitoso y login rechazado.	Evidencias para documentación de seguridad.
Integrante 3	Configurar en Postman el uso del token como variable global. Verificar que se pueda reutilizar en VehicleService.	Variable de token en Postman.	Base lista para proteger VehicleService.
Integrante 4	Documentar el flujo inicial de autenticación JWT y preparar diagrama de flujo.	Borrador del diagrama JWT.	Material para documentación técnica final.

Endpoint mínimo	Propósito	Resultado esperado
POST /auth/login	Autenticar usuario con credenciales válidas.	Datos del usuario y token JWT.

Entregables obligatorios del Integrante 1	Descripción
AuthService funcional	Debe permitir login únicamente a usuarios existentes, activos y con contraseña correcta.
JWT	Debe generarse en el login y usarse para consumir otros servicios.
Bcrypt	Debe aplicarse para validar contraseñas cifradas.
Evidencias	Capturas de login exitoso, login incorrecto y usuario inactivo.
Entrega al equipo	Token, credenciales de prueba, formato del header de autorización y explicación breve.

6. Fase 3 - UserService

Responsable principal: Integrante 2. Esta fase debe dejar usuarios administrables y un mecanismo de validación para que otros servicios consulten si un usuario existe y está activo.

Integrante	Actividades y obligaciones	Entregable individual	Entregable para el siguiente
Integrante 1	Apoyar con validación del JWT en UserService para mantener seguridad uniforme.	Filtro, utilería o criterio común de validación de token.	Seguridad lista para UserService.
Integrante 2	Programar registro, edición, perfil y cambio de estatus. Validar rol administrador, duplicados, campos obligatorios y clave única.	UserService funcional con pruebas exitosas y de error.	Endpoint estable de validación de usuario para VehicleService y ParkingService.
Integrante 3	Probar que usuarios existentes puedan usarse para asociar vehículos.	Confirmación de idUsuario válido para pruebas.	Datos reales para registrar vehículos.
Integrante 4	Documentar endpoints de UserService en la tabla de API REST y guardar capturas relevantes.	Tabla de endpoints y evidencias de usuarios.	Material listo para documento técnico.

Endpoint mínimo	Función	Validación clave
POST /usuarios	Registrar usuario.	Solo administrador, correo único, username único, contraseña cifrada.
PUT /usuarios/{idUsuario}	Editar usuario.	No editar usuario, contraseña ni clave de usuario.
GET /usuarios/{idUsuario}	Ver perfil.	Token válido e identificador obligatorio.
PATCH /usuarios/{idUsuario}/estatus	Cambiar estatus.	Solo administrador y sin eliminación física.
GET /usuarios/validar/{claveUsuario}	Validar usuario para otros servicios.	Usuario existente y activo.

Validación obligatoria	Evidencia esperada
Token obligatorio	Solicitud sin token rechazada.
Solo administrador registra usuarios	Usuario normal intentando registrar debe fallar.
Correo único	Registro con correo repetido debe fallar.
Username único	Registro con username repetido debe fallar.
Clave única generada internamente	Usuario registrado devuelve o permite consultar clave de usuario.
Cambio de estatus	Usuario pasa de activo a inactivo o viceversa sin eliminarse.

7. Fase 4 - VehicleService

Responsable principal: Integrante 3. Esta fase debe dejar vehículos administrables y un endpoint de validación para que ParkingService compruebe placas, estatus y asociación con el usuario.

Integrante	Actividades y obligaciones	Entregable individual	Entregable para el siguiente
Integrante 1	Verificar que VehicleService use la misma lógica de JWT que los demás servicios.	Servicio protegido de forma uniforme.	Validación de token reutilizable.
Integrante 2	Proporcionar usuarios válidos, claves de usuario y estatus para asociar vehículos.	Lista de usuarios disponibles para pruebas.	idUsuario y claveUsuario para registros de vehículos.
Integrante 3	Programar búsqueda, registro, edición y cambio de estatus. Validar placa única y límite de cuatro vehículos activos.	VehicleService funcional con casos exitosos y errores.	Endpoint estable de validación de vehículo para ParkingService.
Integrante 4	Probar vehículos desde Postman y preparar evidencias para documentación.	Capturas y resultados de pruebas.	Datos listos para probar entrada y salida.

Endpoint mínimo	Función	Validación clave
GET /vehiculos/usuario/{idUsuario}	Buscar vehículos de un usuario.	Token válido e idUsuario obligatorio.
POST /vehiculos	Registrar vehículo.	Placa única, vehículo activo por defecto y máximo cuatro activos.
PUT /vehiculos/{idVehiculo}	Editar vehículo.	Solo vehículos asociados al usuario autenticado.
PATCH /vehiculos/{idVehiculo}/estatus	Cambiar estatus.	Solo vehículos asociados al usuario autenticado.
GET /vehiculos/validar	Validar placa e idUsuario para ParkingService.	Vehículo activo y perteneciente al usuario.

Validación obligatoria	Evidencia esperada
Token obligatorio	Consulta o registro sin token debe fallar.
Placa única	Registro de placa repetida debe fallar.
Máximo cuatro vehículos activos	Intento de quinto vehículo activo debe fallar.
Vehículo activo por defecto	Registro nuevo debe quedar activo.
Solo editar vehículo propio	Intento de editar vehículo ajeno debe fallar.
Cambio de estatus	Vehículo se inactiva sin eliminarse.

8. Fase 5 - ParkingService e integración

Responsable principal: Integrante 4. Esta fase es la más delicada porque integra usuarios, vehículos, cajones, movimientos, tiempos y costos. Debe demostrar que los microservicios se comunican sin compartir bases de datos físicamente.

Integrante	Actividades y obligaciones	Entregable individual	Entregable para el siguiente
Integrante 1	Apoyar con consumo REST entre servicios y manejo de token para llamadas internas o protegidas.	Cliente REST o criterio de consumo entre servicios.	ParkingService puede consultar UserService.
Integrante 2	Garantizar que el endpoint de validación de usuario responda correctamente.	Validación estable de usuario activo.	ParkingService puede validar claveUsuario.
Integrante 3	Garantizar que el endpoint de validación de vehículo responda correctamente.	Validación estable de placa, estatus y pertenencia.	ParkingService puede validar placa e idUsuario.
Integrante 4	Programar entrada, salida, consulta de espacios, cálculo de tiempo/costo y actualización de cajones.	ParkingService funcional e integrado.	Flujo completo listo para Postman y demostración.

Endpoint mínimo	Función	Validación clave
POST /parking/entrada	Registrar entrada de vehículo.	Usuario activo, vehículo activo, asociación correcta, máximo dos vehículos dentro y cajón disponible.
PUT /parking/salida	Registrar salida de vehículo.	Movimiento abierto, usuario activo, vehículo asociado, cálculo de costo y liberación de cajón.
GET /parking/espacios	Consultar espacios.	Token válido y datos de espacios disponibles.

Regla obligatoria	Acción que debe ocurrir	Evidencia esperada
Usuario activo	Validar usuario antes de permitir entrada o salida.	Usuario inactivo rechazado.

Regla obligatoria	Acción que debe ocurrir	Evidencia esperada
Vehículo asociado al usuario	Comparar placa con el usuario que intenta entrar o salir.	Placa ajena rechazada.
Máximo dos vehículos dentro por usuario	Contar movimientos abiertos por usuario.	Tercera entrada rechazada.
Entrada ocupa cajón	Actualizar cajón como ocupado después de entrada exitosa.	Consulta muestra cajón ocupado.
Salida libera cajón	Actualizar cajón como libre después de salida exitosa.	Consulta muestra cajón disponible.
Costo automático	Calcular minutos, horas cobradas y costo total.	Respuesta de salida muestra costo.

9. Fase 6 - Docker y despliegue

Objetivo: levantar microservicios y bases de datos en contenedores. La distribución recomendada simula tres servidores, respetando los dominios del proyecto.

Integrante	Actividades y obligaciones	Entregable individual	Entregable para el siguiente
Integrante 1	Dockerizar AuthService, UserService y PostgreSQL. Configurar red, puertos y variables de entorno del servidor 1.	Contenedores del servidor 1 levantados.	Auth y usuarios disponibles para pruebas.
Integrante 2	Revisar variables de entorno, credenciales y conexión de UserService a PostgreSQL.	Configuración estable del dominio usuarios.	Servidor 1 validado.
Integrante 3	Dockerizar VehicleService y MySQL vehículos. Validar conexión y scripts.	Contenedores del servidor 2 levantados.	VehicleService disponible para ParkingService.
Integrante 4	Dockerizar ParkingService y MySQL estacionamiento. Probar comunicación con los otros servicios.	Contenedores del servidor 3 levantados.	Sistema completo listo para Postman.

Servidor	Debe contener	Responsable fuerte
Servidor 1	AuthService, UserService y base de datos de usuarios.	Integrante 1 con apoyo del Integrante 2.
Servidor 2	VehicleService y base de datos de vehículos.	Integrante 3.
Servidor 3	ParkingService y base de datos de estacionamiento.	Integrante 4.

10. Fase 7 - Postman y pruebas

Objetivo: crear una colección que funcione como demo guiada. Debe mostrar casos exitosos y errores reales, no solo endpoints aislados.

Integrante	Actividades y obligaciones	Entregable individual	Entregable para el siguiente
Integrante 1	Crear carpeta Auth en Postman. Configurar login exitoso, login incorrecto y guardado del token.	Requests de autenticación probados.	Token automático para las demás carpetas.
Integrante 2	Crear carpeta Usuarios. Probar registro, duplicados, perfil, edición y estatus.	Requests y evidencias de UserService.	idUsuario y claveUsuario para vehículos y parking.
Integrante 3	Crear carpeta Vehículos. Probar registro, placa repetida, quinto vehículo, consulta, edición y estatus.	Requests y evidencias de VehicleService.	idVehiculo y placa para ParkingService.
Integrante 4	Crear carpeta Parking y ordenar flujo final de demostración.	Colección final exportada.	Demo lista para exposición.

Orden de pruebas recomendado	Qué demuestra
Login administrador	JWT funcional y credenciales correctas.
Login incorrecto o usuario inactivo	Validaciones de seguridad.
Registrar usuario	UserService y permisos de administrador.
Registrar usuario repetido	Validación de duplicados.
Registrar vehículo	VehicleService y asociación con usuario.
Registrar placa repetida o quinto vehículo	Validaciones del dominio vehículos.
Consultar espacios	ParkingService y base de estacionamiento.
Registrar entrada	Integración usuario + vehículo + espacio.
Registrar salida	Cálculo de tiempo/costo y liberación de cajón.
Caso de error de parking	Reglas de negocio críticas.

11. Fase 8 - Documentación técnica

Objetivo: llenar la plantilla técnica con información real del proyecto, evidencias, diagramas, endpoints y conclusiones. La documentación debe ser clara, no extensa innecesariamente.

Integrante	Apartados o evidencias a entregar	Obligación individual
Integrante 1	Introducción, objetivos, arquitectura general, AuthService, JWT, bcrypt, despliegue servidor 1, diagrama JWT.	Entregar explicación técnica de seguridad y arquitectura.

Integrante	Apartados o evidencias a entregar	Obligación individual
Integrante 2	UserService, endpoints de usuario, reglas de negocio, capturas de pruebas, casos exitosos y errores.	Entregar evidencias y explicación de validaciones de usuarios.
Integrante 3	VehicleService, endpoints de vehículos, reglas de negocio, capturas de pruebas, casos exitosos y errores.	Entregar evidencias y explicación de validaciones de vehículos.
Integrante 4	ParkingService, integración, despliegue, Postman final, evidencias generales, conclusiones y documento integrado.	Integrar el documento final y revisar consistencia.

Apartado de la plantilla	Responsable recomendado
Portada	Integrante 4
Introducción y objetivos	Integrante 1
Explicación de microservicios	Todos, cada quien su servicio
Arquitectura del sistema y arquitectura por capas	Integrante 1 con apoyo de todos
Diagramas	Integrante 1 e Integrante 4
Diseño de API REST	Todos, cada quien sus endpoints
Tecnologías de desarrollo	Integrante 1
Tecnologías de almacenamiento	Integrantes 2 y 3
Tecnologías de despliegue	Integrante 4
Pruebas, casos exitosos y casos de error	Todos
Evidencias	Todos
Conclusiones	Todos, integradas por Integrante 4

12. Fase 9 - Ensayo de exposición y defensa

Objetivo: asegurar que todos puedan explicar su parte y que el flujo completo funcione antes de la demostración oficial.

Integrante	Debe explicar	Debe demostrar
Integrante 1	Arquitectura general, JWT, bcrypt, AuthService y contenedores del servidor 1.	Login exitoso, token y protección de endpoints.
Integrante 2	UserService, reglas de usuario, rol administrador, clave única y estatus.	Registro, perfil, edición y cambio de estatus.
Integrante 3	VehicleService, placas, límite de cuatro vehículos y asociación con usuario.	Registro de vehículo, error por placa repetida y consulta por usuario.
Integrante 4	ParkingService, integración REST, entrada, salida, espacios y cálculo de costo.	Entrada, ocupación de cajón, salida, liberación y costo total.

Guion de demostración	Responsable sugerido
Mostrar estructura del proyecto y contenedores levantados.	Integrante 1
Mostrar bases separadas por dominio.	Integrantes 1, 3 y 4
Hacer login y obtener token.	Integrante 1
Registrar usuario, consultar perfil y cambiar estatus.	Integrante 2
Registrar vehículo, consultar vehículos y probar placa repetida.	Integrante 3
Consultar espacios, registrar entrada y mostrar cajón ocupado.	Integrante 4
Registrar salida, mostrar cajón liberado y costo total.	Integrante 4
Mostrar un caso de error relevante.	Cada integrante en su servicio
Cerrar con conclusiones y aprendizajes.	Todos

13. Cadena de entregables entre integrantes

Esta tabla es la más importante para evitar bloqueos. Cada integrante debe entregar una pieza consumible por el siguiente, no solamente su código.

De quién	Para quién	Debe entregar
Integrante 1	Integrante 2	Estructura base, conexión a PostgreSQL, JWT funcional, login administrador, token de prueba y criterio de seguridad.
Integrante 2	Integrante 3	Usuarios válidos, idUsuario, claveUsuario, estatus y endpoint de validación de usuario.
Integrante 3	Integrante 4	Vehículos válidos, placas activas, idVehiculo y endpoint de validación de vehículo.
Integrante 4	Todo el equipo	Flujo completo de entrada/salida, colección Postman final y evidencias generales.
Todo el equipo	Integrante 4	Capturas, endpoints, explicación de cada servicio, casos exitosos y casos de error.
Integrante 4	Todo el equipo	Documento integrado, paquete final y guion de demostración.

14. Entregable individual obligatorio

Para demostrar participación real, cada integrante debe conservar una carpeta o sección con sus evidencias. Esto ayuda en la revisión y en la defensa oral.

Elemento individual	Descripción
Servicio asignado	Código fuente del microservicio o parte técnica correspondiente.

Elemento individual	Descripción
Endpoints probados	Lista de endpoints implementados y su propósito.
Requests y responses	Ejemplos de entradas y salidas, sin saturar el documento con código.
Capturas de Postman	Evidencia de casos exitosos y casos de error.
Explicación técnica	Resumen corto de qué hizo, cómo funciona y qué reglas de negocio cumple.
Aportación documental	Texto, tablas o evidencias entregadas para la documentación final.

15. Checklist final por integrante

Integrante 1 - Arquitectura/Auth	Debe estar terminado
AuthService	Login funcional.
JWT	Token generado y reutilizable.
Bcrypt	Contraseña validada de forma segura.
Pruebas	Login exitoso, login fallido y usuario inactivo rechazado.
Docker	Dockerfile y contenedores del servidor 1.
Documentación	Arquitectura general, AuthService y diagrama JWT.

Integrante 2 - Usuarios	Debe estar terminado
UserService	Registro, edición, perfil y estatus.
Validaciones	Admin, correo único, username único y campos obligatorios.
Clave de usuario	Generada internamente y única.
Endpoint de validación	Disponible para ParkingService.
Pruebas	Casos exitosos y errores documentados.
Documentación	Tabla de endpoints y evidencias de usuarios.

Integrante 3 - Vehículos	Debe estar terminado
VehicleService	Registro, búsqueda, edición y estatus.
Validaciones	Placa única, máximo cuatro activos y vehículo propio.
Endpoint de validación	Disponible para ParkingService.
Docker	Dockerfile y contenedores del servidor 2.
Pruebas	Casos exitosos y errores documentados.
Documentación	Tabla de endpoints y evidencias de vehículos.

Integrante 4 - Parking/Integración	Debe estar terminado
ParkingService	Entrada, salida y consulta de espacios.
Integración	Consulta a UserService y VehicleService.
Reglas de negocio	Usuario activo, vehículo asociado, máximo dos dentro, ocupar/liberar cajón.
Costos	Minutos, horas cobradas y costo total.
Postman	Colección final exportada y ordenada.
Documentación	Documento final integrado con evidencias y conclusiones.

16. Orden de ejecución recomendado

Día o bloque	Trabajo recomendado	Resultado esperado
Día 1	Fase 0 y Fase 1: estructura, bases y datos de prueba.	Todos pueden comenzar sin bloquearse.
Día 2	Fase 2: AuthService, JWT y seguridad base.	Token listo para proteger servicios.
Día 3	Fase 3 y Fase 4: UserService y VehicleService en paralelo.	Usuarios y vehículos funcionales.
Día 4	Fase 5: ParkingService e integración.	Flujo de entrada/salida funcional.
Día 5	Fase 6 y Fase 7: Docker y Postman.	Sistema levantado y colección lista.
Día 6	Fase 8: documentación con evidencias.	Documento técnico completo.
Día 7	Fase 9: ensayo de exposición y corrección final.	Demo segura y equipo preparado.

17. Criterio de cierre de cada fase

Una fase solo se considera cerrada cuando cumple los cinco elementos siguientes:

1. Código o configuración funcional.
2. Prueba realizada en Postman o evidencia equivalente.
3. Captura o registro de evidencia.
4. Explicación breve para documentación.
5. Entrega clara para que el siguiente integrante continúe.

18. Conclusión operativa

El proyecto debe organizarse como una línea de ensamblaje: Auth valida, UserService confirma usuarios, VehicleService confirma vehículos y ParkingService registra entrada/salida. La meta no es crear funciones extra, sino demostrar el flujo completo con seguridad, bases separadas, comunicación REST, contenedores, pruebas y documentación clara.

Ruta final del equipo: Autenticación -> Usuarios -> Vehículos -> Estacionamiento -> Postman -> Documentación -> Demo.